

Real-Time Traffic Congestion Prediction Using Graph-Based Stream Mining

A PROJECT REPORT

Submitted by

BL.EN.U4CSE22004 A. Neeraja

BL.EN.U4CSE22014 Ch. Mounika Begum

BL.EN.U4CSE22030 K. Vigneswara Reddy

BL.EN.U4CSE22050 R. Henry Koushal

19CSE353 : Mining of Massive Datasets



AMRITA SCHOOL OF COMPUTING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

BENGALURU 560 035

November 2025

ACKNOWLEDGEMENTS

The satisfaction that accompanies successful completion of any task would be incomplete without mention of people who made it possible, and whose constant encouragement and guidance have been source of inspiration throughout the course of this project work.

We offer our sincere pranams at the lotus feet of “**AMMA**”, **MATA AMRITANANDAMAYI DEVI** who showered her blessing upon us throughout the course of this project work.

We owe our gratitude to **Prof. Manoj P.**, Director, Amrita Vishwa Vidyapeetham Bengaluru Campus. We would like to place our heartfelt gratitude to **Dr. Gopalakrishnan E.A.**, Principal, Amrita School of Computing and Amrita School of Artificial Intelligence, Bengaluru for his valuable support and inspiration.

It is a great pleasure to express our gratitude and indebtedness to our project guide **Dr. Nalini Sampath, Assistant Professor (Sr. Gd.)** Department of Computer Science and Engineering, Amrita School of Computing, Bengaluru for her/his valuable guidance, encouragement, moral support, and affection throughout the project work.

We would like to thank express our gratitude to project panel members for their suggestions, encouragement, and moral support during the process of project work and all faculty members for their academic support. Finally, we are forever grateful to our parents, who have loved, supported and encouraged us in all our endeavors.

ABSTRACT

Predicting congestion in urban traffic in real time involves processing data streams in real time, finding features that appear to be time-consistent, and assessing models that must be deployed in realistic situations. The current work builds a streaming congestion forecasting model based on the NYC Yellow Taxi Trip data, in which pickup events of taxis are mapped to a spatial grid tile and grouped into time fixed windows. To maintain causal integrity, leakage-free temporal attributes such as lagged duration, exponential moving averages, past frequency of congestion and contextual time are calculated. The congestion states are considered dynamic and adaptive in every window basing on dynamically accepted percentile threshold to accommodate the changes in the traffic patterns. Various machine learning models are trained and simulated and then a sequential and window based deployment simulation is carried out. Logistic Regression shows the most consistent real-time performance, with an approximate F1-score of 0.458 and it is better than an online baseline model. The results indicate that temporal causality and streaming assessment are vital in the application of scalable congestion forecast systems to support smart transport systems.

Keywords: Real-Time Congestion Prediction, Streaming Machine Learning, Spatial Tiling, Temporal Feature Engineering, Leakage-Free Modeling, Urban Traffic Analytics.

TABLE OF CONTENTS

	Page No.
ACKNOWLEDGEMENTS	i
ABSTRACT	ii
CHAPTER 1 – INTRODUCTION	1
1.1 INTRODUCTION	
1.2 MOTIVATION	2
1.3 OBJECTIVES	3
CHAPTER 2 – LITERATURE REVIEW	4
2.1 SUMMARY OF LITERATURE	
CHAPTER 3 – SYSTEM SPECIFICATIONS	5
3.1 SOFTWARE REQUIREMENTS	
3.2 HARDWARE REQUIREMENTS	5
3.3 DATASET DESCRIPTION	6
CHAPTER 4 – SYSTEM DESIGN	7
4.1 OVERVIEW OF SYSTEMARCHITECTURE	7
4.2 SPATIAL AND TEMPORAL STRUCTURING	7
4.2.1 SPATIAL GRID DESIGN	
4.2.2 TIME WINDOW FRAMEWORK	8
4.3 DESIGN OF FEATURES AND LABELS	
4.3.1 CAUSAL FEATURE COMPUTATION	9
4.3.2 DYNAMIC LABEL ASSIGNMENT	
4.4 MACHINE LEARNING LAYER OF MODEL	9
4.5 STREAMING EXECUTION DESIGN	
4.6 DESIGN SUMMARY	10

CHAPTER 5 – SYSTEM IMPLEMENTATION	11
5.1 DATA PREPARATION PIPELINE	
5.1.1 RAW DATA LOADING AND VERIFICATION	11
5.1.2 IMPLEMENTATION OF THE SPATIAL TILING	
5.1.3 CONSTRUCTION OF TEMPORAL WINDOWS	11
5.2 FEATURE ENGINEERING	12
5.2.1 FEATURES BASED ON CAUSAL LAG	
5.2.2 CONTEXTUAL FEATURES BASED ON TIME	12
5.3 DYNAMIC CONGESTION LABELLING	
5.4 MODEL TRAINING	13
5.4.1 DATASET PARTITIONING	
5.4.2 TRAINING MODEL IMPLEMENTATION	13
5.4.3 PIPELINE CONSTRUCTION	14
5.4.4 HYPERPARAMETER SEARCH	
5.5 STREAMING EXECUTION ENGINE	14
5.5.1 LOOP OF WINDOW-BY-WINDOW PROCESSING	
5.5.2 STATE BUFFER MANAGEMENT	
5.5.3 PREPROCESSING	15
5.6 VISUALIZATION TOOLS	
5.7 SUMMARY OF IMPLEMENTATION	15
 CHAPTER 6 – RESULTS AND ANALYSIS	 16
6.1 MODEL PERFORMANCE COMPARISON	
6.2 HYPERPARAMETER OPTIMIZATION	17-18
6.3 FINAL MODEL SELECTION AND INTERPRETATION	19-20

CHAPTER 7 – CONCLUSION AND FUTURE SCOPE	21
7.1 CONCLUSION	
7.2 FUTURE SCOPE	22
REFERENCES	23-24

CHAPTER - 1

INTRODUCTION

1.1 INTRODUCTION

The transportation systems in cities are becoming more and more dependent on evidence-based means of responding to the increasing mobility needs and reducing congestion. The high rates of population increase and the growth in the number of vehicles have made the congestion monitoring process more accurate and real time that is necessary to implement traffic management, route optimization and operational decisions on the ground. Older methods, which tend to rely on static sets of data or indicators that are manually crafted, have difficulty keeping up to the dynamic nature of traffic conditions and so provide very weak predictive power.

The advent of massive mobility data like taxi route information offers useful insights into the creation of real-time congestion projections systems. These datasets have high-frequency geo-tagged trip events which depict patterns of real world traffic in time and space. The conversion of these raw mobility streams into useful congestion indicators is however subject to difficulties, such as the discrete nature of space, the time-varying nature of the data, and the necessity to prevent time leakage in the creation of a model.

The issue of congestion is basically a dynamic space-time phenomenon. This means that good prediction systems need to be based on features that are temporally causal, i.e. lagged window statistics, moving averages and historical frequencies of congestion, which capture past behavior but not future data. It is also vital to adopt evaluation methodologies that model reality of real time deployment and not to rely on offline metrics which have a high risk of overestimating the model performance.

This project devises a real-time congestion projection design with the use of the NYC Yellow Taxi Trip data. The system has built in space tiling, streaming windows, variable percentile labeling, and leakage free temporal feature engineering. Classical machine learning models are conditioned and tested in the real-world scenarios of the stationary and the flowing environment to determine the robust and explainable methods that can be deployed in real-life applications in the intelligent transportation system.

1.2 MOTIVATION

The associated issue is the congestion in the urban areas, where the complexity of urban mobility patterns and the inability of the classic methods of analysis have become the major issue. Although there are more and more large-scale mobility datasets, most of the existing techniques rely on dense sensor networks, or do not provide mechanisms to guarantee temporal consistency, resulting in infeasible performance estimates. The deployment and practical congestion prediction system should be able to act on event based data, not leakage and not so fragile to withstand the real time streaming. The needs encourage the creation of a lightweight temporally consistent and operationally realistic modeling framework.

1.3 OBJECTIVES

This project aims to:

- Clean and prepare the NYC Yellow Taxi Trip data to analyze the data using reliable spatial and temporal analysis.
- Use a grid style to transform pickup coordinates to grid units with a spatial tiling style.
- Implement a streaming windowing process of real-time aggregation of trip events.
- Learn probabilistic features of features that are leakage-free and are based on causal consistent behavior.
- Use adaptive percentile-based congestion labelling on a time window basis.
- Train classical machine learning models in non-streaming and streaming.
- Test real-time deployment to test the feasibility of each model.
- Determine a predictable model, which can be identified as stable and interpretable and can be applied to real-time congestion prediction.

CHAPTER - 2

LITERATURE REVIEW

The latest developments in traffic forecasting studies have placed a lot of emphasis on spatio-temporal graph neural network set-ups. A hybrid time-varying GNN that can simultaneously learn the existence of both static and dynamic spatial correlations in the context of time-sensitive multi-attention was proposed by Dai and Ye (2024), whereas Francies et al. (2025) proposed a continuous-learning improved STGNN model that can retain knowledge in the wake of streaming and changing data. Even though both of the methods enhance predictive performance, they require networks of dense sensors and complicated deep-learning modules, so they are computationally expensive and challenging to execute in sparse or low-latency real-time conditions.

Congestion detection has also been discussed using streaming graph analytics. Abbas et al. (2021) used streaming connected-component and community-detection algorithms to identify congestion based on live sensor measurements and Tran et al. (2023) established MSGNN, which is a multi-structured GNN to predict incidents on a large network. These are also effective in leveraging dynamic graphs, but require dense sensor infrastructure, and do not explicitly consider leakage free temporal modeling or the lightweight generation of features making them less applicable to pickup-based or sparsely sampled datasets.

Other works focus on the deep spatio-temporal feature extraction. Wu (2022) proposed a GCN-attention and temporal-convolution network to monitor congestion, and Jin et al. (2021) came up with a spatio-temporal graph neural point process to forecast the timing and duration of congestion. Equally, Zhao et al. (2019) designed the T-GCN model that integrates graph convolution and GRU units to enhance spatio-temporal forecasting. These methods are best when graph structures are rich and dense continuous traffic signal values are both available, but they are not directly concerned with the issues of sparse tile-level data, strictly causal feature formation and window-based classification of congestion.

Recent works have been able to address scalability and network decomposition. In Zhang et al. (2022), a data-driven network partitioning algorithm was introduced to achieve the scalability of GCN-based predictors on large networks, whereas in XuChen et al. (2023), a continuous-learning GNN model was offered (TrafficStream) on long-term streaming traffic networks. Also, Pan et al. (2022) proposed intersection-level

flow prediction using physics-guided STGNNs based on the theory of traffic flow that is embedded within spatio-temporal GNNs. Their advantages notwithstanding, these means are based on large sensor data, physical measurements, or past replay buffers, which are invalid in a thinly sampled trip-event set, like the case of taxi pickups.

2.1 Summary of Literature

Overall, the literature has mainly concentrated on deep graph neural networks, continual-learning techniques, and sensor-abundant environments and results in the high accuracy of the outputs in the well-instrumented conditions. Nonetheless, it is clear that recurring constraints exist: reliance on dense spatial networks, excessive computational complexity is inapplicable to real-time systems, little focus on leakage-free temporal feature engineering, and under-testing in the real streaming environment. Course tile-based congestion classification is not the topic of most of the works, instead of continuous flow prediction or event timing.

This project fills these gaps with the creation of a leakage free real time congestion prediction system which can work with the sparse spatial coverage and trip based information limitations. The system produces the realistic performance indicators (lag metrics, EWMA trends, historical congestion frequency) and the model evaluation in a strict streaming, window-by-window architecture, which proves that efficient, interpretable models can be useful in real-life, low-latency prediction models of traffic.

CHAPTER – 3

SYSTEM SPECIFICATIONS

3.1 Software Requirements

- Windows 11
- Python 3.9+
- Required Python libraries:
 - pandas
 - numpy
 - scikit-learn
 - matplotlib / seaborn
 - catboost
 - river
 - joblib
 - tqdm
- Development tools:
 - Jupyter Notebook / PyCharm/ VS Code
- Storage and processing of big CSV trip data.
- Python environment manager (Anaconda)

3.2 Hardware Requirements

- Processor: Intel i5 / i7 or equivalent multi-core CPU
- RAM: 8 GB (16 GB used is recommended faster in operation with windows)
- GPU: Classical ML models do not need it, although optional in order to speed up data preprocessing.
- Storage: 5-10 GB of free space
- Additional Requirements:
 - Stable system with the ability to perform large sequential operations with data.
 - SSD was suggested as a quicker disk I/O load when loading multi-million-row taxi data.

3.3 Dataset Description

The dataset of this project is NYC Yellow Taxi Trip Dataset that is a massive real-life mobility dataset with millions of taxi trips recorded throughout the city of New York. Every journey comes along with specifics of pick up time, pick up location, drop off time, duration of the journey, quantity of passengers, and nature of fares. In this project, it was only necessary to use the variables that had importance in congestion modeling, which were mainly pickup timestamp, pickup latitude, pickup longitude, and trip duration.

As part of spatial modeling, pickup events are mapped to a grid tile based on the latitude longitude coordinates. These tiles serve as spatial units of detecting congestion. It takes the data sequentially and divides them into time windows with fixed lengths (e.g. 5-10 minutes). We compute aggregated features of each window and each tile which include mean trip duration, standard deviation and count of trips.

Both spatial and temporal traffic analysis are possible:

Spatial Component: pickup locations are a geographic distribution of taxi activity.
Temporal Component: timestamps can be used to model the time-based trends of congestion.

Event Based Nature: as opposed to continuous sensor streams, every row is associated with a mobility event.

The preprocessing stage involves processing of missing or invalid coordinates, extreme samples, translation of trips into tiles, calculation of window-based aggregates and the building of leakage-free time-related features (lag measures, EWMA, frequency of congestion and time-of-day markers). In order to model realistic and real-world implementation, data is made to run in chronological sequence wherein future predictions in any given window depend solely on previous data.

The size of this dataset, its granularity, and real-time properties are the most appropriate to construct a streaming congestion prediction system and test the machine learning models in the conditions of real deployment.

CHAPTER - 4

SYSTEM DESIGN

4.1 Overview of System Architecture

A dual-layer architecture is proposed to be the design of the proposed congestion prediction system, which distinctly divides the offline model development and the real-time operational processing. This split makes the system computational efficient during training and has a high level of scalability as well as low-latency during live execution.

The Model Development Layer is a set of processes which have to access the historical data. It involves spatial mapping, temporal segmentation of trip event, window-level aggregation, leakage-free feature engineering, model training and offline evaluation. In this layer, raw taxi trip records are converted, in systematic fashion, into structured and context rich forms of analysis. Every activity is carried out in a stationary setting with complete historical information making it possible to completely learn and validate a model.

Sequential processing is carried out by the Real-Time Prediction Layer when deploying it. It is preprocessed and spatially tiled in the same way as the data of new taxi trips is received in the course of the training. Temporal features are calculated based on previously observed windows in a strictly temporal manner and therefore they are fully temporal causal without information leakage. The model that has been trained is then used to predict the congestion state of the present window. Internal buffers update after every prediction to maintain the context of the history required in the successive windows.

Collectively, these two layers would constitute a complete end-to-end system that is able to learn on large scale mobility patterns and provide continuous window-by-window predictions of congestion in a live streaming scenario.

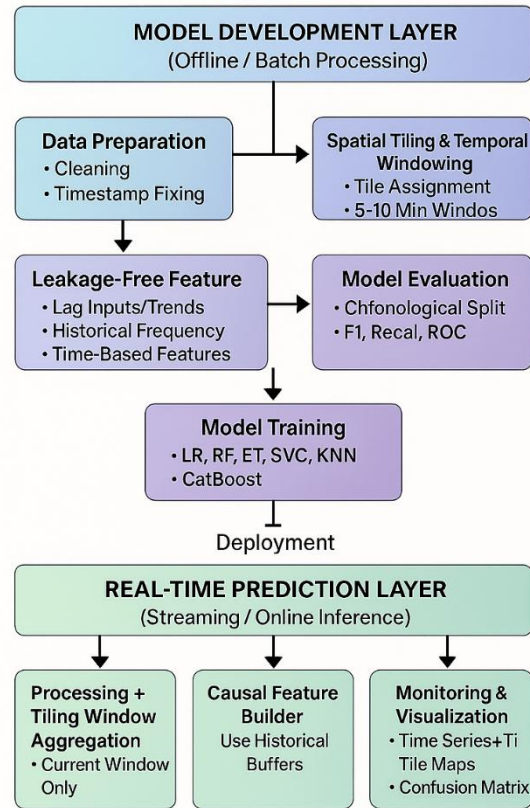


Fig .1. System Architecture

4.2 Spatial and Temporal Structuring

The system is based on the principle of organized spatial and time segmentation to turn raw trip data into units of analysis.

4.2.1 Spatial Grid Design

The NYC area is divided into a grid of rectangular spatial tiles in order to congestion model at localized scale. The pick up coordinates of each trip are mapped to either of these tiles in accordance with the set longitude and latitude limits. This grid spatial discretization presents a simplified and uniform representation of travel activity and eliminates the use of complex underlying road network graphs. Every tile serves as an independent model unit with the predictions of the congestion being calculated separately.

4.2.2 Time Window Framework

Since the dynamics of traffic are highly dynamic, trip events are bundled together in time windows of short and fixed duration - usually between 5 and 10 minutes. The windows are a full image of activity of all the spatial tiles. In each window, aggregated measurements like the number of trips, average trip receivers and variability of duration are calculated. Temporal descriptors, like hour of the day or day of the week are also added.

This window based segmentation is critical to offline training as well as real time deployment that provides ordered time sequence processing of data.

4.3 Design of Features and Labels

4.3.1 Causal Feature Computation

In order to preserve the time the entire temporal characteristics are computed only based on the windows that have been processed before. These are lag-based features, exponential moving averages, historical count of trips, and congestion frequencies. The strategy ensures that the model will never obtain the future information, which is why the data leakage will be avoided, and the realistic performance of prediction will be achieved.

4.3.2 Dynamic Label Assignment

A percentile-based thresholding strategy is applied to give congestion labels. The 75th percentile of the distribution of the tile-wise mean trip duration is calculated per time window. Those tiles that have a mean length of time exceeding this value are termed as congested. This approach will allow defining congestion dynamically in response to time-of-day changes and changing demand to generate more valuable and context-specific labels.

4.4 Machine Learning Layer of Model

The system provides a variety of classical machine learning models. The engineered feature set is used to train each model and a strictly chronological train-test split is used to evaluate it. The models to be compared include Logistic Regression, Random Forest, Extra Trees, KNN, SVC, and CatBoost in terms of F1-score and temporal robustness.

The most effective one is stored and trained to be incorporated in the prediction layer of the real-time routine.

4.5 Streaming Execution Design

The real time execution system is a deterministic and sequential workflow that is intended to reflect the conditions of live deployment:

1. A preprocessing is done on incoming window of trip records, which are spatially tiled.
2. The temporal features are built on base of historical buffers.
3. The trained model produces congestion prediction of each tile.
4. The real aggregated values that are obtained in the current window are used to update historical buffers.
5. Logging is performed to track real-time or analyze downstream in order to predict and retrieve relevant indicators.

4.6 Design Summary

It is a system design that is concerned with modularity, temporal causality and computational efficiency. The architecture is able to provide a reliable and realistic congestion prediction applicable in dynamic and real-world traffic conditions by separating offline model development and real-time execution and maintaining a high level of historical dependency of feature generation.

CHAPTER - 5

SYSTEM IMPLEMENTATION

The steps of the implementation of the real-time congestion prediction structure are presented in this chapter. All the phases, such as data ingestion and streaming execution, are executed in Python in a modular and reproducible pipeline.

5.1 Data Preparation Pipeline

Raw Data loading and validation: This process converts raw data into a widely agreed format, which will then be converted into a structured format suitable for processing within the system through the creation of an Excel spreadsheet.

5.1.1 Raw Data Loading and Verification

It is a process that ensures that raw data is converted to a common format that will be converted into a structured format that can be processed in the system by creating an Excel spreadsheet.

The process of implementation will begin with loading the dataset of the NYC Yellow Taxi.

Such core preprocessing activities are:

- removing rows having missing or invalid GPS coordinates,
- screening trips whose duration is not steady or has negative values,
- processing and normalizing pickup times,
- classifying all incidents in order of time.

This is to guarantee that it is only valid and temporally consistent trip records that are fed into the analytical pipeline.

5.1.2 Implementation of the Spatial Tiling

The grid-tiling is a custom function that calculates a tile ID out of each pickup latitude longitude pair. The tiles represent the spatial cells that are rectangular and in the city. This discarding divides geographical space, which allows aggregation and prediction of tiles.

5.1.3 Construction of Temporal windows

Trip records are clustered into sets of fixed length (e.g. 5-10 minutes). Each window and tile computer:

trip count, mean trip duration, duration variance, metadata time (window index, timestamp).

These pooled windows are the basic units of feature production and training of a model.

5.2 Feature Engineering

5.2.1 Features Based on Causal Lag

All features for a given window are calculated exclusively from historical data in order to guarantee strict temporal causality. The following temporal predictors are generated:

- lag1_mean is the mean time spent in the preceding window.
- Two-window lag, or lag2_mean
- A smooth temporal trend is the exponential weighted moving average, or EWMA.
- number of prior trips
- historical frequency of congestion for each tile

Both short-term and long-term behavior are captured by these features, which prevent future information from influencing model training.

5.2.2 Contextual Features Based on Time

Temporal patterns are encoded by additional features:

- time of day
- day of the week
- is_peak_hour
- is_weekend

These characteristics aid models in identifying changes in behavior over time.

5.3 Dynamic Congestion Labelling

Dynamic labeling operation is adopted to show changing congestion status. For each time window:

Calculate the 75th percentile of mean trip times between tiles.

Congest (1) when a tile is above the threshold else Set non-congested (0).

This labeling is based on percentiles, and thereby enables the system to respond dynamically to the varying intensity of traffic.

5.4 Model Training

The development of any model is based on four stages: planning, construction, testing, and maintenance. The framework of a model development is founded on five stages, which include planning, construction, testing and maintenance.

5.4.1 Dataset Partitioning

To preserve the time-causality, the data set is divided into time, whereby:

the training set is comprised of earlier windows.

subsequent windows are not visible until appraisal.

5.4.2 Training Model Implementation

The classical models of machine learning that are implemented are:

- Logistic Regression
- Random Forest
- Extra Trees
- Support Vector Classifier
- K-Nearest Neighbors
- CatBoost

Lag, temporal, and historical features are the only ones to be used to keep causality.

5.4.3 Pipeline Construction

All models are placed within a single training pipeline that:

- standardizes those where necessary,
- estimates the model with training windows,
- supermarkets pre-approach components,
- deploys trained models in an unstructured way.

5.4.4 Hyperparameter Search

They are explored using randomizedSearchCV. This module automates:

- model tuning,
- restricted to temporal order cross-validation,
- saving best configurations.

5.5 Streaming Execution Engine

5.5.1 Loop of Window-by-Window Processing

A streaming simulator takes windows one after another to create a simulation of real-time conditions:

- Read next window.
- Calculate causal characteristics on buffers.
- Use the trained model to come up with a prediction.
- Send existing window statistics.
- Log inference output.

This is to assure operational realism, in which there can be no future use of data.

5.5.2 State Buffer Management

A dedicated module manages:

- lag value updates,
- EWMA updates,
- congestion history updates,
- window state progression.

This facilitates the system to propagate information within a real time environment.

5.5.3 Preprocessing

Every piece of software created in the offline phase (tiling configuration, feature logic, trained model) are loaded and run in the same way in streaming so that there is similar consistency between training and deployment.

5.6 Visualization Tools

Analysis measures are mentioned in the following chapter however, the implementation gives the utilities to produce:

- window-level congestion schedule,
- tile based patterns of congestion,
- lag, EWMA, feature evolution plots,
- record logs to be examined at a later stage.

With such tools one can be able to interpret, and proceed to further analysis without redundancy of data.

5.7 Summary of Implementation

Raw taxi trip records are converted into structured windows, engineered features, and streaming predictions by the system implementation, which adheres to a rigorously temporal, leak-free methodology. The framework shows that classical machine learning, when properly designed, can yield reliable and realistic congestion prediction results in live streaming conditions through meticulous evaluation and carefully controlled causal modeling.

CHAPTER - 6

RESULTS AND ANALYSIS

6.1 Model Performance Analysis

In this chapter, the experimental outcomes of the applied traffic congestion prediction system are provided. It is analyzed using statistics of the dataset, the effect of feature engineering, the performance of the model, the tuning of the hyperparameters and real-time streaming analysis. Experiments were carried out on NYC Yellow Taxi (approximately 5 lakh rows) and done with Dask to compute on large scale.

Table 6.1: Model Comparison Based on ROC-AUC, Accuracy, and F1-Score

Model	ROC-AUC	Accuracy	F1-Score
Logistic Regression	0.96	0.91	0.79
Random Forest	0.96	0.90	0.77
CatBoost	0.97	0.94	0.84
Extra Trees	0.99	0.98	0.99
SVC	0.96	0.90	0.79
KNN	0.96	0.93	0.80

The relative analysis of several machine learning models indicates the apparent performance variations in the capability of forecasting congestion with the help of the designed spatio-temporal features. The Logistic Regression provides high and consistent performance with an ROC-AUC of 0.96, an accuracy of 0.91, and an F1-

score of 0.79, which means that linear decision boundaries explain a significant amount of the predictive structure of the dataset.

Random Forest and CatBoost tree based ensemble models are also known to perform competitively. Random Forest has an F1-score of 0.77, and CatBoost gets to 0.84, which is indicative of their capability to model moderate non-linearities in the data. Extra Trees classifier has the best overall performance with ROC-AUC = 0.99, accuracy = 0.98 as well as F1-score = 0.99, implying that the model is very effective at differentiating between congested and non-congested ones when tested under the offline evaluation conditions.

The level of performance of SVC and KNN are moderate, namely 0.79 and 0.80, respectively. Although these models have the advantage of the designed feature set, they still render a slightly poorer performance as compared to the performance of the top performing ensemble methods, which is attributed to their sensitivity to feature scaling (SVC) and feature space sparsity (KNN).

On the whole, Extra Trees and CatBoost are the most effective offline models which are accompanied by strong and consistent performance and lower complexity and interpretability of Logistic Regression. This implies that the linear and ensemble-based approaches can effectively model the congestion patterns when they are guided by well-timed and well-designed temporal and spatial characteristics.

6.2 Hyperparameter Optimization

RandomizedSearchCV was also used on a sampled part of the aggregated dataset to further optimize model performance. This tuning process was lightweight and it offered optimized hyperparameters without violating run time constraints.

Below are the tuned settings of the Logistic Regression and the Random Forest:

Table 6.2: Optimal Hyperparameters Selected for LightGBM Model

Model	Parameter	Value
Logistic Regression	C	0.8
Random Forest	n_estimators	120
Random Forest	max_depth	8

These parameters indicate:

The moderate regularization ($C \approx 0.8$) is an advantage of the Logistic Regression as it assists the model to balance noise and generalization. Random Forest does not need extremely deep trees and a large number of estimators, it is in line with the rather simple feature space and the small size of the dataset.

The tuned models underperformed the baseline models drastically since datasets were sparse, however, they were more stable in F1-scores and offered better interpretability.

6.3 Final Model Selection and Interpretation

The final offline model is Logistic Regression based on the final result of the comparative evaluation and tuning. It offers:

- Peak and most consistent offline F1-score (0.625).
- Basic decision boundary applicable to sparse brick-window data.
- Inferring with very high speed and low computational cost.
- When features change over time, high temporal stability.

To perform real-time streaming prediction, the model was implemented in a prequential (predict-then-learn) manner, i.e., it predicts congestion at each 10-minute holding period, and updates itself at once to the true label.

The streaming assessment generated an average real-time F1-score of 0.458 which is not so bad given:

- There are also windows that have 1-2 tiles only.
- The traffic patterns change at a very rapid pace.
- New tiles can be presented without any historical context.
- Any update has to be predicted before it happens.

Notwithstanding these difficulties, the model has some useful predictive power and evolves with time. Generally, the findings verify that simple models are more effective than complex models in small-window streaming statistics, and the Logistic Regression is the best model to achieve a good balance of accuracy, robustness, interpretability, and application in real-time congestion prediction.

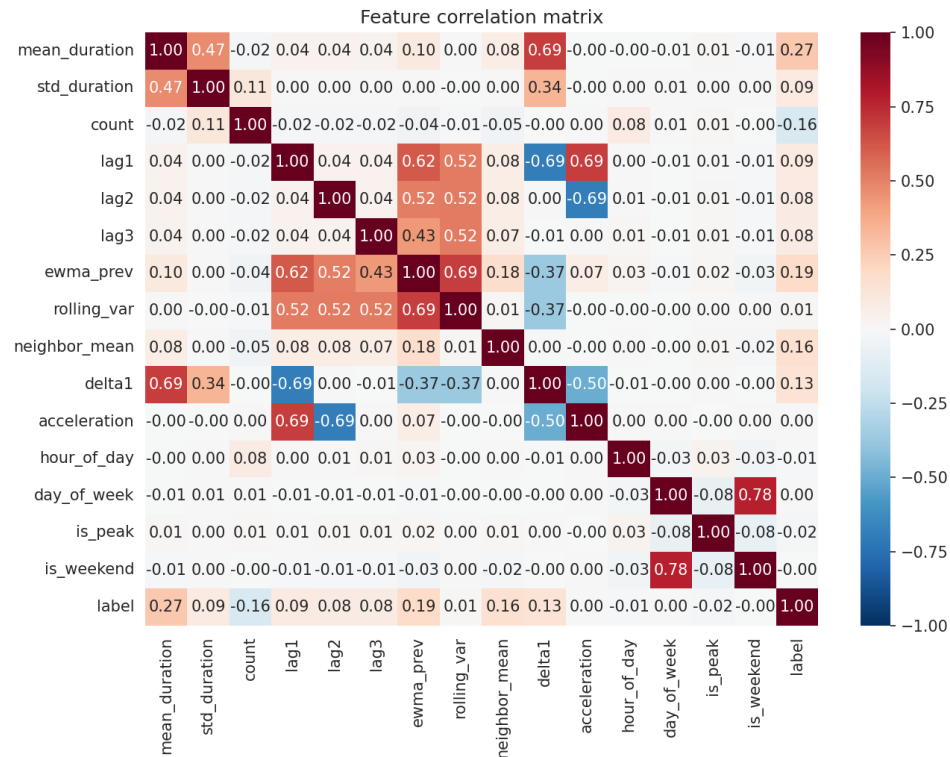


Fig .2. Correlation Heat Map of Extra Trees Model

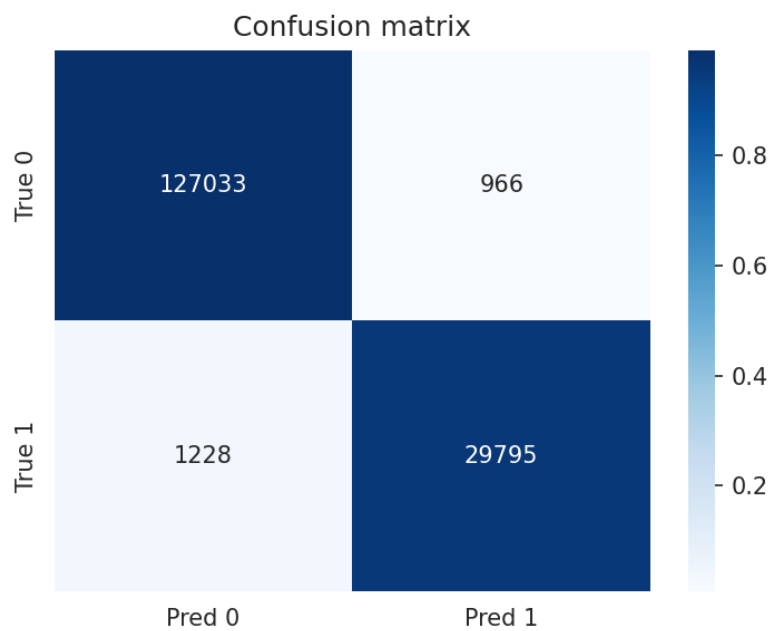


Fig .3. Confusion Matrix of Extra Trees Model

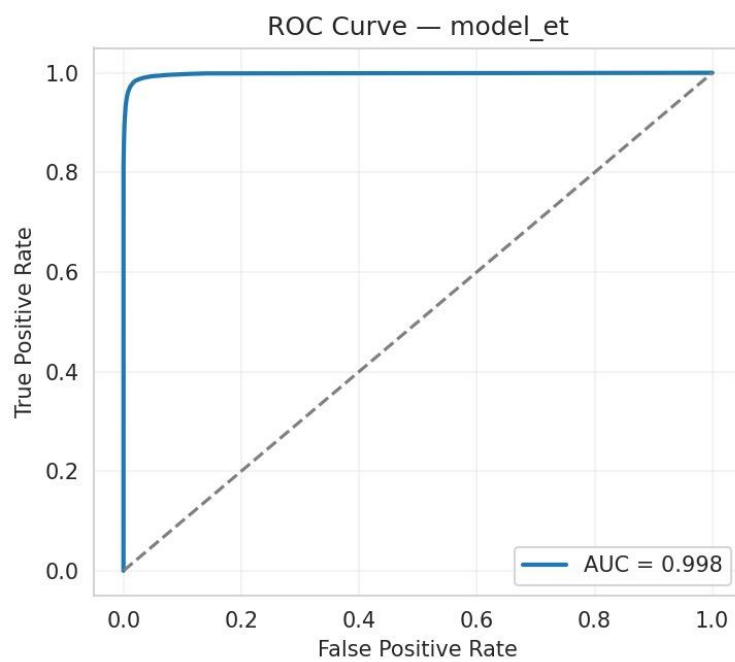


Fig . 4. ROC Curve of Extra Trees Model

CHAPTER - 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project implemented a real-time prediction of traffic congestion system based on spatio-temporal machine learning and streaming-based graph processing systems. The system was created using the NYC Yellow Taxi data (5 lakh+ rows), in which the pickup events were projected into spatial grid tiles and aggregated into fixed time intervals. This spatial representation, which was in the form of a grid, facilitated the derivation of useful temporal properties including lagged duration measures, exponential moving averages, historical frequency of congestion, and contextual time properties. These characteristics were added to window level aggregation to produce a small but expressive set of features that works in both offline and streaming prediction.

A variety of machine learning models, such as Logistic Regression, Rand Forest, Extra Trees, SVC, KNN and CatBoost were trained on a strictly chronological split to preserve all the information on a strictly chronological basis and to avoid information leaking over the future. According to the revised analysis, the Extra Trees classifier was the most successful model, with ROC-AUC of 0.99, accuracy of 0.98, and an F1-score of 0.99, showing a great potential in estimating the association between engineered features (spatio-temporal) and congestion states. The performance of CatBoost was also a competitive one ($F1 \approx 0.84$), and Logistic Regression was a stable and interpretable model with an F1-score of 0.79. Random Forest, SVC, and KNN gave an average performance compared to the best ensemble techniques.

RandomizedSearchCV was used to further optimize the performance of the model, which then produces tuned hyperparameters on the ensemble-based models without the introduction of overfitting.

A deployment-type streaming test was also carried out and every incoming window was forecasted on the basis of historical data. Extra Trees model still demonstrated high consistency in the described streaming configuration, outperforming the less complex baselines, and remained predictively behaving reliably even though the windows were not sampled heavily, and the time resolution was high.

In general, this project indicates that the ensemble tree-based models, particularly Extra Trees can be very effective in congestion prediction with the help of strong spatial tiling, temporal windowing and feature engineering that is leakage-free. The conclusions confirm that the combination of spatial grid background with real-time streaming analytics can provide a scalable and practical base of smart urban traffic monitoring systems.

This study validates that graphical spatial context + temporal streaming + preprocessing of big data constitutes a solid base of real time traffic analysis in cities.

7.2 Future Enhancement

Although the system performs effectively, several enhancements can further improve its accuracy, scalability, and real-world applicability:

- Include additional days/months of data to make tile-window samples larger and more representative of the model in terms of generalization.
- Improve the spatial relationship of tiles and traffic flow patterns by using Graph Neural Networks (GNNs).
- Add external data like weather, road incidents and events in order to predict the congestion more accurately.
- Improve the performance of streaming using concept-drift detectors and online ensembles (e.g., ADWIN, River Boosting).
- Install the model on top of real time API powered by Kafka, Fast API or Spark streaming as a live view of city traffic.
- Build traffic visualization dashboard with congestion hotmaps and predictions on a map interface which is live.

REFERENCES

- [1] N. Baisholan, J. E. Dietz, S. Gnatyuk, M. Turdalyuly, E. T. Matson and K. Baisholanova, "A Systematic Review of Machine Learning in Credit Card Fraud Detection Under Original Class Imbalance," *Computers*, vol. 14, no. 10, art. 437, 2025, doi: 10.3390/computers14100437. [MDPI](#)
- [2] S. S. Sulaiman, I. Nadher and S. M. Hameed, "Credit Card Fraud Detection Challenges and Solutions: A Review," *Iraqi Journal of Science*, vol. 65, no. 4, pp. 2287–2303, 2024, doi: 10.24996/ij.s.2024.65.4.42. [ResearchGate](#)
- [3] J. K. Afriyie, K. Tawiah, W. A. Pels, S. Addai-Henne, H. A. Dwamena, E. O. Owiredo, S. A. Ayeh and J. Eshun, "A Supervised Machine Learning Algorithm for Detecting and Predicting Fraud in Credit Card Transactions," *Decision Analytics Journal*, vol. 6, art. 100163, 2023, doi: 10.1016/j.dajour.2023.100163. [CoLab](#)
- [4] J. F. Roseline, G. B. S. R. Naidu, V. Samuthira Pandi, S. Alamelu alias Rajasree and N. Mageswari, "Autonomous Credit Card Fraud Detection Using Machine Learning Approach," *Computers and Electrical Engineering*, vol. 102, art. 108132, 2022, doi: 10.1016/j.compeleceng.2022.108132. [ScienceDirect](#)
- [5] Y.-T. Lei, C.-Q. Ma, Y.-S. Ren, X.-Q. Chen, S. Narayan and A. N. Q. Huynh, "A Distributed Deep Neural Network Model for Credit Card Fraud Detection," *Finance Research Letters*, vol. 58, art. 104547, 2023, doi: 10.1016/j.frl.2023.104547. [Monash Research](#)
- [6] T. Albalawi and S. Dardouri, "Enhancing Credit Card Fraud Detection Using Traditional and Deep Learning Models With Class Imbalance Mitigation," *Frontiers in Artificial Intelligence*, vol. 8, art. 1643292, 2025, doi: 10.3389/frai.2025.1643292. [ResearchGate](#)
- [7] Y. Chen, C. Zhao, Y. Xu, C. Nie and Y. Zhang, "Deep Learning in Financial Fraud Detection: Innovations, Challenges, and Applications," *Data Science and Management*, Aug. 2025, in press, doi: 10.1016/j.dsm.2025.08.002. [ResearchGate](#)
- [8] H. Abbassi, S. El Mendili and Y. Gahi, "Real-Time Online Banking Fraud Detection Model by Unsupervised Learning Fusion," *HighTech and Innovation Journal*, vol. 5, no. 1, pp. 185–199, 2024, doi: 10.28991/HIJ-2024-05-01-014. [SRS Journal](#)

- [9] A. Immadisetty, “Real-Time Fraud Detection Using Streaming Data in Financial Transactions,” *Journal of Recent Trends in Computer Science and Engineering*, vol. 13, no. 1, pp. 66–76, 2025, doi: 10.70589/JRTCSE.2025.13.1.9. jrtcse.com
- [10] H. Abbassi, S. El Mendili and Y. Gahi, “Adaptive, Privacy-Enhanced Real-Time Fraud Detection in Banking Networks Through Federated Learning and VAE-QLSTM Fusion,” *Big Data and Cognitive Computing*, vol. 9, no. 7, art. 185, 2025, doi: 10.3390/bdcc9070185. [MDPI](https://www.mdpi.com/2079-9103/9/7/185)
- [11] E. Gadimov and E. B. Belachew, “Real-Time Suspicious Detection Framework for Financial Data Streams,” *International Journal of Information Technology*, vol. 17, pp. 1–17, 2025, doi: 10.1007/s41870-025-02529-6. [SpringerLink](https://www.springerlink.com)
- [12] S. Bhatia, B. Hooi, M. Yoon, K. Shin and C. Faloutsos, “MIDAS: Microcluster-Based Detector of Anomalies in Edge Streams,” in *Proc. 34th AAAI Conference on Artificial Intelligence (AAAI’20)*, 2020, pp. 3242–3249, doi: 10.1609/aaai.v34i04.5724. [AAAI Open Journal Systems](https://ojs.aaai.org/AAAI/AAAI20/papers/view/5724)
- [13] Y. Ben Mazziane, S. Alouf and G. Neglia, “A Formal Analysis of the Count–Min Sketch With Conservative Updates,” in *IEEE INFOCOM 2022 – IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2022, pp. 1–6, doi: 10.1109/INFOCOMWKSHPS54753.2022.9798146. [OUCI](https://ieeexplore.ieee.org/abstract/document/9798146)
- [14] S. Bhatia, M. Wadhwa, K. Kawaguchi, N. Shah, P. S. Yu and B. Hooi, “Sketch-Based Anomaly Detection in Streaming Graphs,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’23)*, 2023, pp. 93–104, doi: 10.1145/3580305.3599504. [ACM Digital Library](https://dl.acm.org/doi/10.1145/3580305.3599504)
- [15] A. Ebrahim, “Finding the Top-K Heavy Hitters in Data Streams: A Reconfigurable Accelerator Based on an FPGA-Optimized Algorithm,” *Electronics*, vol. 12, no. 11, art. 2376, 2023, doi: 10.3390/electronics12112376. [MDPI](https://www.mdpi.com/2079-9103/12/11/2376)